index_watch.py — guide

Revision: 1 **Last modified:** 2026-09-25T02:20:00Z

Script: `scripts/codegraph/index_watch.py` · tests: `scripts/codegraph/tests/test_watch_startup_race.py` (3 cases) · mutation harness: `scripts/codegraph/tests/test_watch_startup_race_mutations.sh` (golden + 3 mutants).

Overview

Progress-proven monitor for a running `codegraph index` / `init` / `sync` bulk job (constitution §11.4.232(C)/(E), §11.4.201, §11.4.273 — every reading is a measurement from the live process, never inferred from liveness alone). Reads `.codegraph/codegraph.lock` for the indexer's PID, confirms via real `/proc/<pid>/cmdline` (never a bare `pgrep`), samples `/proc/<pid>/io` + per-thread CPU + the live sqlite DB's row counts + the indexer's own log tail, and classifies each sample as `STARTING` / `PROGRESSING` / `STALL_HOT` / `STALL_IDLE` / `NOT_RUNNING`.

`codegraph_safe.sh` (the sanctioned single-writer entry point) launches this as one of its supervised bulk-run processes; it is also safe to run manually (`python3 index_watch.py --project <dir>`) against an already-running indexer for independent monitoring.

Prerequisites

`python3` (stdlib only). Run from anywhere — `--project` is an explicit argument, not assumed.

Usage

```
python3 scripts/codegraph/index_watch.py --project <dir> [--once] [--
interval N] [--stall-min M]
    [--json-out FILE] [--quiet]
```

exit	meaning
0	last verdict was `PROGRESSING` / `STARTING` (loop still running unless `--once`)
3	last verdict was a `STALL_*`
4	`NOT_RUNNING` and the indexer never completed
5	`NOT_RUNNING` and `db.index_state == "complete"`

2026 09 25

Cold-start grace (fixed - -)

Bug (found live): the loop exits the instant ANY sample reports `NOT_RUNNING`. On a genuine cold start the supervisor backgrounds this watcher at essentially the same instant it launches the indexer — if the watcher's first sample lands before the indexer has written `.codegraph/codegraph.lock`, `read_lock_pid()` returns `None`, `alive` is `False`, verdict is `NOT_RUNNING`, and the watcher exits forever, one sample after being born, having never observed the indexer — leaving a live, multi-hour bulk run with **zero** stall detection. Reproduced live: an auto-launched watcher logged exactly one line (`NOT_RUNNING ... pid: null ... db=0.00GB`) and was gone while the indexer itself ran healthily for hours afterward.

Fix: `STARTUP_GRACE_SAMPLES` (default 3) — a bounded number of consecutive not-alive samples, counted only while the indexer has never yet been observed alive, are classified `STARTING` (retryable) instead of terminal `NOT_RUNNING`. This never re-arms once `alive` has genuinely been observed even once — a real stop after a real run is still terminal on the very next sample (no fresh grace mid-run). A

genuinely-absent indexer (never starts at all) still resolves `NOT_RUNNING` once the grace window is exhausted — the fix is additive-only, it does not mask a permanent absence.

Edge cases and guarantees

- PID liveness is proven via real `/proc/<pid>/cmdline` containing `"codegraph"`, never a bare `pgrep` substring match (constitution §11.4.196(D)/§12.12 — a carrier process merely mentioning the string would false-match).
- Progress is proven from multiple independent signals (file/node counts growing, DB size growing, the indexer's own log-progress line advancing, `io.write_bytes` increasing) — any one advancing counts as progress, so a phase transition that temporarily quiets one signal doesn't false-trigger a stall.
- State is persisted to `.codegraph/index_runs/watch_state.json` (atomic temp-then-rename) so a watcher restart can resume `prev` from the last sample — this also means a *stale* state file from an unrelated earlier run on the same project directory carries forward its `ever_alive` / `not_alive_streak` fields; this is pre-existing behaviour of state-file reuse, not something the grace-window fix introduces or changes.

Related

`codegraph_safe.sh` (the supervisor that launches this), `disk_tripwire.sh` (the sibling disk-safety monitor), `fk_cascade_probe.py` / `fk_index_patch.py` (the bulk-window FK-cascade fix this watcher's progress signal helped diagnose), constitution §11.4.201 / §11.4.232(C)(E) / §11.4.273.

Last verified

2026-09-25: cold-start-race regression test 3/3 cases GREEN across 3 repeated runs; mutation harness golden PASS

- M1–M3 KILLED across 3 repeated runs (identical). Live-verified against a real full-repo `codegraph index` run (584,328 files) — manually-launched watcher tracked it correctly from `STARTING` through `PROGRESSING` past the 54% mark (the point a prior, unrelated run had previously stalled at) with no false stall/not-running reports.